CardanoRnR — System Architecture Update

Milestone 3 Supplement to Technical Design Document

Project: Open-Source Review and Reputation System — Hotel Use Case

Fund 12 | Project ID: 1200173

Purpose

This document supplements the original CardanoRnR Technical Design Document submitted in Milestone 1. It describes the architectural changes introduced in Milestone 3 in response to reviewer feedback. The original design document remains unchanged and can be read alongside this update.

---

1. Data Layer — Blockchain Network (updates Section 2.2.6)

Previous design

Review data hashes were stored as immutable proofs on-chain. The State UTxO was held at the operator enterprise address.

Updated design

The State UTxO is now held exclusively at the script address. On every review submission, the transaction consumes the current State UTxO and produces a new one at the same script address. This brings the system in line with the standard Cardano UTxO state machine pattern.

A one-shot NFT (State Thread Token) is minted at system initialisation by consuming a specific genesis UTxO. This NFT travels with the State UTxO at all times. The off-chain service filters UTxOs by this NFT before reading reputation state — any UTxO at the script address that does not carry the NFT is ignored. The on-chain validator enforces that the NFT must appear in the continuing output on every spend, preventing it from leaving the script address through a review transaction.

- NFT Policy ID: 2f64f96cc3aaec4afa662a84538d2edc4f854b387895d46305e16134
- Token name: RnrStateToken
- Minting: one-shot, consuming genesis UTxO — cannot be re-minted
- State UTxO location: script address (not operator wallet)

2. Smart Contract Specifications (updates Section 2.4)

Previous design

The contract exposed storeReview(), validateData(), and ensureAuthorizedUser() as its primary interface.

Updated design

The contract has been restructured around three operations:

lockReview()

Locks the review datum at the script address. Accepts the review content, rating, booking reference, and reviewer identity. This is the first on-chain transaction in the review flow and is called synchronously during the HTTP request.

processReview()

Consumes both the locked review UTxO and the current State UTxO. Produces an updated State UTxO at the script address carrying the NFT forward. The updated datum reflects the new cumulative rating count and total score used for reputation calculation. This transaction is built and submitted by the background worker after lockReview() completes.

The on-chain validator (validNFTForwarded) checks that the NFT appears in the continuing output on every processReview() call. The business operator private key signs the transaction, which is the authorisation mechanism for this trusted-operator model.

fetchLatestChainState()

Off-chain function that queries UTxOs at the script address and returns the one carrying the State Thread Token. Filters by NFT first, then reads ratingCount and totalScore from the inline datum. This replaces the previous approach of selecting the UTxO with the highest ratingCount, which was vulnerable to spoofing via a crafted UTxO.

---

3. Application Layer — Blockchain Queue Service (addition to Section 2.2.3)

An asynchronous queue service sits between the HTTP review endpoint and the on-chain transaction builder. The queue is implemented using Bull with a Redis backend.

When a review is submitted via the API:

- lockReview() is called synchronously and the lock transaction hash is returned to the caller immediately
- The job (lock hash, review data, booking ID) is added to the Bull queue
- The HTTP response is returned — the user sees confirmation without waiting for blockchain processing
- A background worker picks up the job and calls processReview(), which builds and submits the state update transaction
- Once processReview() completes, the review record in MongoDB is updated with the redeem transaction hash, booking ID, and reputation score

This decoupling prevents HTTP timeouts during periods of network congestion on the Cardano testnet and allows the system to retry failed processReview() attempts without losing the review.

---

4. reviewId Generation (update to review uniqueness)

Previous design

reviewId was derived from the user identifier alone, meaning a returning guest who booked twice would produce the same reviewId for both reviews.

Updated design

reviewId is now generated from the combination of userId and bookingId:

```
reviewId = Buffer.from(`${user._id}${booking._id}`).toString("hex")
```

Each review is now uniquely identified by who submitted it and which booking it is associated with. The bookingId is also stored in the on-chain datum (field 1) and in the MongoDB review record, providing an auditable link between the booking system record and the on-chain review.

5. reviewReferenceId

reviewReferenceId is now generated from the bookingId:

```
reviewReferenceId=new Constr(0,
[Buffer.from(bookingId).toString("hex")])
```

The original Technical Design Document (CardanoRnR_TechDesign.pdf) remains the primary reference. This document records the delta introduced in Milestone 3 only.